

Session 2 · Platforms, agents, multimodal RAG and knowledge graphs

English · Student hands-on workbook

Author

Rajkumar Rajagobalan

Use this workbook alongside the executable Colab notebook. Follow the link in the opening section. Record observations on paper or in your saved notebook.

Two hours · Labs 5–8 · Intermediate-to-expert Python · Google Colab + OpenAI

Name / pair: _____ Date: _____

[Open in Colab](#)

Build a small assistant for a **fictional university makerspace**. All policies, room slots, charts, people and graph facts are original synthetic classroom data. This session extends the evidence and guardrail concepts from Session 1, but runs in a fresh notebook without its variables.

The source chapters are 5, 7, 8 and 9. Here they become workshop **Labs 5, 6, 7 and 8**. The platform implementation uses OpenAI managed retrieval, adapting the Vectara chapter's lifecycle concepts. The graph implementation uses a small in-memory graph. It does not install Neo4j or Microsoft GraphRAG.

Outcomes and pacing

Clock	Activity	Evidence to record
00–10	Setup and preflight	READY, model names, resource recovery plan
10–35	Lab 5: RAG platform	Index readiness, filtered results, cited answer
35–60	Lab 6: AI agent	Actual tool trace and boundary tests
60–65	Break	Saved notebook and resource ledger
65–85	Lab 7: multimodal RAG	Retrieved image, caption-only/image comparison
85–110	Lab 8: knowledge graph	Two-hop evidence, hybrid candidates, revocation
110–120	Review, cleanup and submission	Notebook, JSON report, cleanup confirmation

For each lab, **predict** → **run** → **inspect** → **explain**. Record measured output in the tables by editing a Colab text cell or adding a Text cell. The final reflection cell exports strings; text-cell worksheets are preserved only in your saved notebook. Do not replace observations with expected answers. Failures are valuable when diagnosed.

Setup · 00–10 minutes

Step 0.1 — Save and configure

1. Open the Colab notebook and select **Copy to Drive**. Name your copy `RAG_Session2_<name-or-pair>`.

2. Use Python 3 and CPU; no GPU, server or extra account is needed.
3. In **Secrets**, add `OPENAI_API_KEY` and enable notebook access. Your project needs the configured models and Files / Vector Stores access.
4. Read the setup code below, then run it once. Look for `READY` and the preflight answer. API usage is billed to your project.

Step 0.2 — Understand what persists

`api` is a small standard-library HTTP client. It accepts only workshop endpoints at the fixed OpenAI origin, records attempts before sending, and times successful and failed requests. It does not log authentication headers. A 45-second timeout and a 100-request classroom limit bound individual requests and the current in-memory counter. DELETE requests remain available after the limit so cleanup can proceed. There are no automatic retries.

`embed` batches new strings, restores API row order, checks 1,536 dimensions and normalizes each vector. The key `(model, exact_text)` prevents reuse across different models or changed text. `grounded` sends evidence IDs with the question and optionally image pixels. It requests the fields `answer`, `abstain`, `sources`. `check_answer` verifies field types and permitted source IDs; **it does not verify that a claim follows from the source**. Human review remains necessary.

`session2_resources.json` records only this workshop's run ID, vector-store ID and file IDs. It is updated after each successful creation, allowing interrupted setup to resume without recreating resources already recorded. A same-runtime rerun preserves counters; a runtime reset loses them. The local file may survive a kernel restart but disappears with a discarded VM. Download it after Lab 5. Never put the API key in this file, code, screenshots or submissions.

The vector store is created with a one-day inactivity expiry. **Original uploaded Files objects need separate deletion**. A notebook request limit is not an account spending limit. `store=False` controls Responses storage, not account-wide data retention.

Step 0.3 — Resolve a setup failure

For 401 check the secret; for 403/404 check project/model/Files permissions; for 429 check quota and rate limits. If the second setup attempt fails, pair with a working student and label shared or recorded observations honestly. Do not rerun an ambiguous resource-creation timeout blindly: use the recovery instructions at the end. Labs 6–8 need setup and the `POLICIES` definitions, but do not call the hosted platform.

Record: `READY` _____ Models _____ Where will you keep the resource ledger? _____

`grounded` reports `guardrail_status`: `contract_passed`, `abstained`, `output_rejected`, `api_refusal` or `incomplete`. Rejected structured output is retained as `raw_output` for diagnosis and must not be treated as an accepted answer. For example, an abstention that still cites a source is rejected. Network and HTTP failures still raise an explicit error. A `READY` message confirms the API calls completed; inspect the preflight status as well.

Lab 5 · A managed RAG platform

10–35 minutes · Source Chapter 5 · 5-minute walkthrough, 15-minute experiment, 5-minute review

Concept: what the platform owns

A managed retrieval service handles the uploaded text's parsing, chunking, embeddings and indexed search. You still own source quality, eligibility rules, answer instructions, evaluation, permissions and lifecycle decisions. This is a managed retrieval backend plus an explicit generation request; it is not a claim of feature equivalence with Vectara's full platform or its hallucination-correction API.

Step 5.1 — Inspect the four policy records

Find current P01 and archived P02. The former allows two hours; the latter allowed four. P03 governs equipment certificates; P04 directs support requests. Predict which policy a question containing “four hours” might retrieve.

Step 5.2 — Upload, attach and wait for readiness

Run the following cell. `upload_policy` builds a multipart form with a file name prefixed by the unique run ID. A Files object stores bytes; attaching it to a vector store starts indexing and supplies attributes `source_id` and `status`.

`prepare_platform` checks the ledger before creating anything. The polling loop reads `file_counts` at most 12 times, with three seconds between pending checks. A slow request can add network time. If it reports `indexing_pending`, rerun the cell; do not clear the ledger. Stop after two such attempts and use a partner's platform output to protect class time. `failed` indexing requires investigation, not a query against an incomplete corpus.

Download `session2_resources.json` when prompted and keep it until cleanup is confirmed. A store's existence does not prove every file is ready.

Record: completed files _____ status _____ ledger downloaded _____

Step 5.3 — Compare results with and without a filter

Read `platform_search`. It asks the provider for up to four results, applies a file-attribute filter when requested, and preserves the source ID, status, score and returned passage. These results are **provider chunks**; unlike Session 1's deduplicated evaluator, this function does not promise k unique documents.

Run the next cell. It performs unfiltered and current-only searches for `Can a student book a makerspace room for four hours per day?`, then generates from the filtered passages.

Review	Observation
IDs / statuses without filter	_____
IDs / statuses with filter	_____
Is P02 absent from filtered evidence?	_____
Generated answer and cited IDs	_____
Does the claim agree with current P01?	_____

Step 5.4 — Explain the result

Change and rerun: save the four-hour result, change `PLATFORM_QUESTION` to `What qualification does a team need to use equipment?`, and rerun the search/generation cell. Inspect whether P03 now supports the answer. Record both questions and their cited IDs in your text worksheet and final reflection; `observations['lab5']` retains only the latest run. This adds API requests.

Record one responsibility moved to the provider and one retained by the application. Scores are not probabilities of factual correctness. The metadata filter uses classroom attributes supplied by trusted code; it is not a multi-tenant authorization system. A valid source ID does not prove the answer's hours are correct.

Checkpoint: four files ready; no archived result in the filtered list; answer reviewed against P01. If P02 did not appear in unfiltered search, record that actual result rather than inventing a rank.

Reflection to save later: `reflections['lab5']` should include IDs, the actual hours stated, and the responsibility split.

Lab 6 · A bounded AI agent

35–60 minutes · Source Chapter 7 · 5-minute walkthrough, 15-minute experiment, 5-minute review

Concept: the model requests; the application executes

The agent loop sends a question and tool definitions to the model, validates a requested function call, executes an allowed function, sends its result back with the matching `call_id`, and lets the model continue. A function call is a request for the application to act. It is not Python code to execute with `eval`.

Step 6.1 — Read the two read-only tools

`search_policy` is a deliberately simple local lexical RAG tool over current policies. Its word-overlap ranking is not the hosted platform from Lab 5. `get_slots` reads a fixed Tuesday availability snapshot. Neither makes reservations. `dispatch` checks names, argument keys and allowed values independently of the model's strict tool schema. This second check protects the actual execution boundary.

`run_agent` allows at most three executed tool calls and four model rounds. `parallel_tool_calls=False` simplifies the trace, but the Python loop still handles every returned call. Each tool output is added to conversation history using the original `call_id`. The final source allowlist contains only evidence actually returned by tools.

Step 6.2 — Predict and run the live loop

For `AGENT_QUESTION`, predict the necessary tools and their likely order. Run the cell. Read `status`, the final answer and every trace entry, including arguments and source IDs. Record what the model actually did; it may choose a different sequence or reach a limit.

Item	Observation
Predicted tool sequence	_____

Item	Observation
Actual tools and arguments	_____
Final status / cited sources	_____
Policy duration and valid Tuesday slots	_____
Did the answer wrongly claim a confirmed booking?	_____

Step 6.3 — Inspect the stopping rule

The budget is checked **before tool dispatch**. The model round that requested the blocked tool may already have cost money. `tool_budget_exhausted`, `round_budget_exhausted`, `tool_rejected` and `output_rejected` are diagnostic outcomes, not successful answers. Trace timings measure local tool execution; they do not include model latency. The API log records request timing separately.

Change and rerun: replace LabA with LabB in AGENT_QUESTION and run the agent cell again. Does `get_slots` receive the new room, and does the answer use its one-hour Tuesday slot? Save both traces before the output is replaced. These are additional live model calls; write both room results in the Lab 6 reflection.

Step 6.4 — Test the execution boundary without API calls

Run the next cell. It rejects an unknown `delete_booking` tool and a Friday request unsupported by the snapshot. Then a **scripted test double**, not an LLM, requests a tool with a zero execution budget. Verify `tool_budget_exhausted`, an empty trace, and no change to `api_log`.

Explain why these deterministic tests prove application behavior but do not prove that a live agent always chooses good tools. The final answer still needs policy and slot review; an allowed source reference does not establish semantic support.

Checkpoint: saved live trace, two local argument/name rejections, zero-budget result. Write the limitation you would test next in reflections ['lab6'] later.

60–65 minutes: break. Save notebook text and output. Keep your resource ledger outside the runtime as well.

Lab 7 · Retrieve an image, then answer from its pixels

65–85 minutes · Source Chapter 8 · 4-minute walkthrough, 12-minute experiment, 4-minute review

Concept: text retrieval can select visual evidence

This is **caption-indexed multimodal RAG**: embed short captions, rank images, then pass the selected image to a vision-capable model. It does not use a shared image/text embedding model. Captions identify the subject but omit the bar values, so finding an image and reading its numbers remain separate tasks.

Step 7.1 — Load verified assets

Run the image-loading cell. It downloads two original PNG charts from the workshop repository and verifies SHA-256 hashes against the release manifest. A hash checks file integrity, not whether a chart's claims are true. `IMAGE_RECORDS` connects a stable image ID, caption and filename. Locally, `SESSION2_DATA_DIR` can point at the downloaded `data` folder.

Read both captions before looking at the pixels. Can either caption answer the exact Tuesday seat count?

Step 7.2 — Predict which image will rank first

`image_search` normalizes embeddings and ranks by dot product. Predict the best source for `How many occupied seats were recorded on Tuesday?`. Run the comparison cell once. The same selected source and question are used for both conditions; the second request additionally receives the PNG as a base64 data URL with `detail='high'`.

Step 7.3 — Compare caption-only and image-assisted answers

The displayed chart is the evidence. Read the Tuesday bar label yourself before checking the model's result. If the wrong chart was retrieved, diagnose retrieval first; do not pretend that adding vision fixes selection.

Condition	Selected source	Actual answer / abstention	Does evidence support it?
Caption only	_____	_____	_____
Caption + pixels	_____	_____	_____

Step 7.4 — Record modality-specific risks

Change and rerun: save the seat comparison, change `IMAGE_QUESTION` to `How many completed print jobs were recorded on Tuesday?`, and rerun. Check whether the retriever switches to `IMG-PRINTS` and read its Tuesday value yourself. Save both source/question comparisons in your worksheet and final reflection; the report otherwise retains only the latest result. New generation requests are billed.

Note the actual number, unit, source ID and any unsupported addition. A valid `IMG-SEATS` source ID alone cannot prove the model read the correct bar. Small labels, ambiguous axes, rotated pages, low resolution and malicious instructions in an image require separate tests. This lab covers image retrieval and visual reading; audio, tables and PDF layout extraction belong in a longer session.

Checkpoint: both ranked IDs, both responses, human-read chart value. Later fill `reflections['lab7']` with the comparison and a new visual test.

Lab 8 · Knowledge graphs and hybrid retrieval

85–110 minutes · Source Chapter 9 · 5-minute walkthrough, 15-minute experiment, 5-minute review

Concept: similarity and relationship constraints answer different questions

A graph records entities and directed, typed relationships. Here a team **HAS_CERT** a certificate, which **QUALIFIES_FOR** a device. Each edge has an ID, status and verification flag. A two-hop path links team to device. Similar wording in a device description cannot grant permission.

The graph is curated teaching data represented as Python dictionaries. This is graph-enhanced RAG with explicit traversal and vector candidates, not Microsoft GraphRAG community summarization, an ontology reasoner or a deployed authorization system. A trusted flag is assumed for the exercise; a production ingestion process must establish it from real evidence.

Step 8.1 — Trace the graph by hand

Read E01–E05. Draw `Team Orion → certificate → device` in a text cell. Label the relationship types and edge IDs. Which edge is revoked? Which is current but unverified? Why must both be excluded?

`eligible_paths` first selects current, verified edges, then traverses only **HAS_CERT** followed by **QUALIFIES_FOR**. Direction and type matter. One hop reaches a certificate and returns no eligible device. Two hops can complete the required relation pattern. This bounded traversal avoids unrestricted model-generated database queries.

Step 8.2 — Compare one hop, two hops and vector candidates

Run the next cell. `graph_candidates` ranks both device descriptions by text similarity; `hybrid` intersects the ranked list with graph-eligible entities. The entire tiny candidate set is ranked here. In a larger truncated candidate pool, graph eligibility does not recover a relevant device that retrieval never considered.

Comparison	Observed result
One-hop eligible devices	_____
Two-hop path edge IDs and terminal device	_____
Vector candidate order	_____
IDs retained after graph constraint	_____
Generated answer and sources	_____

Step 8.3 — Review the generated explanation

The generator receives selected graph-edge statements, current P03 and eligible device text. Check whether its explanation connects the team, certificate and device, rather than just naming the correct device. `check_answer` verifies allowed source IDs, not path completeness in the prose. Compare the actual answer to both path edges and P03.

Step 8.4 — Revoke a relationship and retest

Run the next cell. It deep-copies the graph and revokes E01 without modifying the baseline. The local traversal must now return no eligible device; an unknown team must also return none. A separate paid generation request receives **empty evidence** to test abstention. Distinguish the deterministic graph result from the model's observed behavior.

Change and rerun: replace `revoked[0]['status'] = 'revoked'` with `revoked[0]['verified'] = False` in the control cell. Keep the deep copy, then run it again. This leaves E01 current but unverified: explain why no path is still the correct result. The empty-evidence generation request runs again. Record the revoked and unverified cases separately.

Record: revoked result _____ unknown-team result _____ empty-evidence answer _____

Checkpoint: you can name the exact relationship that changes the result, distinguish current from verified, and explain why a graph can still be incomplete or wrong. Later fill `reflections['lab8']` with evidence IDs and one held-out test, such as a team with two independently valid certificates.

Review and submission · 110–120 minutes

Step 9.1 — Complete your evidence record

Fill all four `reflections` strings and all four `human_review` fields in the next cell, using the tables and actual outputs. Do not leave placeholder sentences. Running this cell saves your strings in memory and makes no API calls. Rerunning it with empty strings clears prior values.

Each lab earns **1 point for measured evidence and 1 for an explanation tied to it**, for 8 points total. Correctly diagnosed failure earns credit. Model success alone is not an interpretation. Prepare a one-minute explanation: *Which layer failed or remains uncertain, what evidence supports that diagnosis, and what new test would evaluate one proposed change?*

Step 9.2 — Delete the resources created for this session

Run the cleanup cell before leaving. It deletes only IDs recorded in your local ledger: first the vector store, then each original uploaded file. It updates the ledger after each successful deletion and accepts 404 as already absent, so an interrupted cleanup can be rerun. It never lists or deletes all resources in the project.

Confirm `complete: True`. If cleanup fails, preserve the ledger and retry after resolving the specific error. Download the updated ledger if anything remains. Do not interpret vector-store expiry as deletion of the separate Files objects. After successful cleanup, Lab 5 needs its creation cell again before reuse; other lab observations remain available for export.

Step 9.3 — Export and submit

Run the export cell. Check both `reflections complete: True` and `cleanup complete: True`. The former checks nonempty fields, not answer quality. Submit `rag_session2_report.json` and your saved `.ipynb`; only the notebook preserves text-cell worksheets. The report includes selected observations and usage, not the key, conversation globals or base64 image payloads.

Recovery and troubleshooting

Problem	Response
Platform indexing pending	Rerun its cell with the same ledger; after two attempts pair and continue
Setup denied Files / Vector Stores access	Ask instructor about project permissions; use paired Lab 5 output and run later labs locally with <code>POLICIES</code> defined
Runtime lost	Restore your downloaded <code>session2_resources.json</code> into Colab Files before setup; rerun definitions and required earlier cells
Timeout during creation	Check the project dashboard for names beginning <code>rag2-classroom-<run_id></code> / <code>rag2-<run_id></code> ; reconcile only this run's resources with the ledger before retrying
Cleanup error	Resolve error, rerun cleanup with the same ledger; manually remove only confirmed workshop IDs if necessary
Image hash mismatch / download error	Use released files; check network or set local data folder; do not disable verification to hide the problem
Output rejected	Inspect response status and allowed sources; record a blocked output separately from a supported answer
Agent exhausts budget	Read the trace; do not increase limits merely to force success

In a fresh runtime, you may run setup, then execute only the `POLICIES` assignment from the platform cell in a new code cell to continue Labs 6–8 when managed indexing is unavailable. Label the platform section as paired/recorded or incomplete. Do not report an unrun lab as completed.

Next steps and references

Return to source Chapter 5 for Vectara; Chapter 7 for agent frameworks and MCP; Chapter 8 for tables, audio and cross-modal embeddings; Chapter 9 for Neo4j, Cypher and broader GraphRAG. The classroom simplifications do not establish production authorization, complete attack resistance, or performance at scale.

See [Python code notes](#), [official references](#), and the instructor's chapter mapping. Keep English code identifiers and test inputs unchanged when using translated notes.